

维度 1

C++ 算法编码工具箱

中文版 - 面试复习与练习手册

定位	维度 1：快速、正确、稳定地把算法思路写成 C++ 代码
重点	STL 算法 / 容器 / iterator / comparator / 二分 / heap / C++20
训练方式	模板掌握 + 小题刻意练习 + 错误模式复盘 + 30 秒自检
难度标记	* 基础必会 ** 面试常用 *** 进阶与高难岗位扩展

核心目标：减少 API 选择错误、边界错误、iterator 错误和 comparator 方向错误。

建议配合 LeetCode / OA / living coding 练习使用。

目录. 学习地图

从工具到题型，再到高频坑和自检。

- 1. 总体框架：维度 1 到底练什么
- 2. 查找与条件判断
- 3. 删除、去重与容器修改
- 4. 排序、Comparator 与稳定性
- 5. 二分查找与有序前置条件
- 6. 选择、分区与 Top-K
- 7. Set / Map / Hash 与集合操作
- 8. Priority Queue 与堆
- 9. Iterator、边界与失效规则
- 10. C++20 / ranges 常用增强
- 11. 33 道维度 1 练习题索引
- 12. 本轮练习暴露出的高频问题
- 13. Tuple 多字段排序与方向包装
- 14. 综合题与面试 30 秒自检

使用方式：第一遍只看 * 和 **；第二遍做练习并对照第 12 章；第三遍再补 ***。

1. 总体框架：维度 1 到底练什么

维度 1 不是“学更多算法”，而是把已有算法稳定地翻译成 C++。

维度 1 的核心能力可以拆成 5 层：选择正确容器 -> 选择正确 STL 算法 -> 写对 comparator / predicate -> 保证 iterator 与边界安全 -> 做复杂度和前置条件检查。

级别	模块	代表内容	面试关注点
*	容器与访问	vector / deque / list / set / map / unordered_* / priority_queue	知道接口、复杂度、iterator category
**	STL Algorithms	find / remove / sort / lower_bound / partition / nth_element	知道前置条件和返回值语义
**	Comparator	strict weak ordering、多字段排序、升降序	避免 <=、方向不一致、取负溢出
**	Iterator	begin/end/rbegin/base/prev/distance	修改容器后判断失效
***	现代 C++	ranges、projection、erase_if、三路比较	减少样板代码，但不能掩盖语义

1.1 面试中真正容易丢分的不是 API 名字

- 算法前置条件不成立：例如 lower_bound 用在未按同一 comparator 排序的范围上。
- 位置边界 off-by-one：第 k 个元素与前 k 个元素的 iterator 不同。
- 修改容器后继续使用旧 iterator。
- predicate / comparator 方向与题意相反。
- 循环某条路径不推进，导致死循环。
- 漏掉空容器、NaN、k 越界、未初始化变量等边界。

训练原则：不要继续堆 API。每道小题做完后强制执行“前置条件 + iterator + 边界 + 方向”四项检查。

2. 查找与条件判断

* find / find_if: 找到第一个匹配元素

- 返回 iterator；没找到返回 end()。
- 解引用前必须先判断 it != end()。
- 需要下标时可用 std::distance(begin, it)，随机访问容器也可 it - begin。

```
auto it = std::find_if(nums.begin(), nums.end(), [](int v) {
    return v > 10 && v % 2 == 0;
});
if (it != nums.end()) {
    auto index = std::distance(nums.begin(), it);
}
```

* all_of / any_of / none_of: 条件聚合

- all_of: 所有元素满足；空范围返回 true。
- any_of: 至少一个满足；空范围返回 false。
- none_of: 没有元素满足；空范围返回 true。

** 反向查找与 reverse_iterator::base()

- rbegin() 从最后一个元素开始。

- 反向 iterator 的 `base()` 指向“反向元素的下一个正向位置”。
- 因此将已找到的 `reverse_iterator` 转成对应正向 iterator，常用 `std::prev(rit.base())`。

```
auto rit = std::find_if(v.rbegin(), v.rend(), pred);
if (rit != v.rend()) {
    auto it = std::prev(rit.base());
}
```

★★ search: 查找连续子序列

- 找到 `pattern` 的第一次出现位置。
- 查找所有出现时，找到后可从 `std::next(it)` 继续，以允许重叠。
- 找不到时必须 `break`，否则 `next` 不变会死循环。

```
auto next = nums.begin();
while (next != nums.end()) {
    auto it = std::search(next, nums.end(), pat.begin(), pat.end());
    if (it == nums.end()) break;
    // process it
    next = std::next(it);
}
```

3. 删除、去重与容器修改

* erase-remove idiom 与 C++20 `std::erase`

- `std::remove` 不改变容器大小，只把“保留元素”搬到前面，并返回 logical end。
- 真正缩小 `vector` 需要 `erase(new_end, end)`。
- C++20 对 `sequence container` 可直接 `std::erase(container, value)`。

```
auto newEnd = std::remove(v.begin(), v.end(), value);
v.erase(newEnd, v.end());

// C++20
std::erase(v, value);
```

* `remove_if` / `erase_if`

- `vector/deque/string` 常见：`remove_if` + `erase`；C++20 可直接 `std::erase_if`。
- `map/set/list/unordered_*` 更适合成员 `erase` 或 `std::erase_if`。

★★ `unique` 只去除“相邻重复”

- 输入 `{1,1,2,2,3,1,1}` 经过 `unique` 后逻辑结果是 `{1,2,3,1}`。
- 全局去重且不关心顺序：`sort` + `unique`。
- 全局去重且保留首次出现顺序：`unordered_set` + 双指针 / 新 `vector`。

★★ 遍历删除时的正确模式

- 关联容器常用 `it = c.erase(it)`，否则 `++it`。
- 不要 `erase(it)` 后再由 `for` 循环 `++it` 使用失效 iterator。

```
for (auto it = scores.begin(); it != scores.end(); ) {
    if (it->second < 60) it = scores.erase(it);
    else ++it;
}
```

高频错误：把 `std::unique(b.begin(), b.end())` 返回的 iterator 传给 `a.erase(...)` 属于不同容器 iterator 混用，行为无效且严重。

4. 排序、Comparator 与稳定性

* `std::sort` 的 comparator 必须是 `strict weak ordering`

- 升序：`a < b`；降序：`a > b`。
- 不能写 `a <= b` 或 `a >= b`，因为 `cmp(x,x)` 必须为 `false`。

- 违反规则后 `std::sort` 的行为未定义。

```
std::sort(v.begin(), v.end(), [](int a, int b) {
    return a < b;    // ascending
});

std::sort(v.begin(), v.end(), std::greater<{}>);
```

★★ 多字段排序：优先明确字段方向

- 例：score 降序、age 升序、name 升序。
- 最稳妥的 living coding 写法是逐字段 if 判断。

```
std::sort(students.begin(), students.end(), [](const auto& a, const auto& b) {
    if (a.score != b.score) return a.score > b.score;
    if (a.age != b.age) return a.age < b.age;
    return a.name < b.name;
});
```

★★ stable_sort：只比较真正的排序 key

- 如果原数组已经按 arrivalSequence 排好，希望相同 priority 保持原顺序，则 comparator 只比较 priority。
- 不要把 arrivalSequence 再放入 comparator，否则元素不再“等价”，stable_sort 的稳定性就失去意义。

```
std::stable_sort(orders.begin(), orders.end(), [](const auto& a, const auto& b) {
    return a.priority > b.priority;
});
```

★★ 避免用 -x 简单表示降序

- 整数 -INT_MIN 无法表示，会发生有符号溢出。
- double 还存在 NaN、+0/-0、infinity 等问题。
- 优先使用显式 comparator；tuple 方向包装见第 13 章。

5. 二分查找与有序前置条件

★ lower_bound / upper_bound / equal_range

- 升序范围：lower_bound 返回第一个 $\geq x$ ；upper_bound 返回第一个 $> x$ 。
- equal_range 返回 $[lower_bound, upper_bound)$ 。
- 出现次数 = `distance(first, second)`。

```
auto first = std::lower_bound(v.begin(), v.end(), x);
auto last  = std::upper_bound(v.begin(), v.end(), x);
auto count = std::distance(first, last);
```

★★ 排序规则与二分规则必须一致

- 如果数据用 `std::greater` 排成降序，lower_bound 也必须传 `std::greater`。
- 二分的本质是依赖范围已经按 comp 划分；不是“随便找一个值”。

```
std::sort(v.begin(), v.end(), std::greater<{}>);
auto it = std::lower_bound(v.begin(), v.end(), x, std::greater<{}>);
```

★★ 异构 lower_bound 与 ranges projection

- 结构体数组按 id 排序时，不必构造完整 User。
- C++20 `ranges::lower_bound` 可以用 projection 直接指定 `&User::id`。

```
auto it1 = std::lower_bound(users.begin(), users.end(), targetId,
    [](const User& u, int id) { return u.id < id; });

auto it2 = std::ranges::lower_bound(users, targetId,
    std::ranges::less{}, &User::id);
```

★★ 成员 lower_bound 优先于通用算法

- set/map 自身的 lower_bound 是树上的 $O(\log n)$ 。
- 对 set iterator 使用 `std::lower_bound` 虽然比较次数可为 $O(\log n)$ ，但 iterator 前进不是随机访问，总体步进可能 $O(n)$ 。

6. 选择、分区与 Top-K

★ partition / stable_partition

- predicate 为 true 的元素被放在前半部分。
- 要求“偶数在前”就写 $v \% 2 == 0$ ；不要把题意方向写反。
- stable_partition 额外保持两组内部相对顺序。

★★ partial_sort: 前 k 个有序

- middle 是半开区间终点，因此前 k 个用 $\text{begin} + k$ 。
- 默认取最小 k 个；取最大 k 个传 `std::greater<>()`。

```
std::partial_sort(v.begin(), v.begin() + k, v.end(), std::greater<>());
```

★★ nth_element: 定位第 k 个元素

- 第 k 个 (1-based) 对应 iterator $\text{begin} + (k - 1)$ 。
- 定位后，左侧元素都不大于/不小于 nth (取决于 comparator)，但左侧本身无序。
- 平均线性复杂度，适合只要第 k 个或 Top-K 无序集合。

```
std::nth_element(v.begin(), v.begin() + (k - 1), v.end(), std::greater<>());
int kthLargest = v[k - 1];
```

最容易混淆：partial_sort 的 middle 是 $\text{begin} + k$ ；nth_element 的 nth 是 $\text{begin} + (k - 1)$ 。

★★ 两次 partition 做三段划分

- 第一次把 $< \text{low}$ 放前面，返回第一段终点。
- 第二次只对剩余范围按 $\leq \text{high}$ 分区，得到中段和大于 high 段。

7. Set / Map / Hash 与集合操作

★ 频率统计: unordered_map

- 典型模板: `++cnt[key]`。
- 需要按频率排序时，把 map 内容搬到 vector 后 sort。

★ 保留原顺序去重: unordered_set

- 见到一个新值才写入 `nums[left++]`；最后 erase 尾部。
- 不能先 sort，因为会改变首次出现顺序。

```
std::unordered_set<int> seen;
size_t write = 0;
for (int x : nums) {
    if (seen.insert(x).second) nums[write++] = x;
}
nums.erase(nums.begin() + write, nums.end());
```

★★ set_intersection 的前置条件

- 两个输入范围都必须按相同顺序排序。
- 如果希望“去重交集”，可先 sort + unique，再 set_intersection。
- 如果直接对含重复的排序数组做 set_intersection，结果会保留 $\min(\text{countA}, \text{countB})$ 个重复。

★★ map 作为“排序 + 聚合”工具

- 例如市场深度: `map<price, qty, greater<>>` 可以天然按 Bid 价格降序聚合。
- Ask 用默认 less 即升序。

8. Priority Queue 与堆

★ priority_queue 默认是大顶堆

- priority_queue<int>: top 是最大值。
- priority_queue<int, vector<int>, greater<>>: top 是最小值。

★★ 第 K 大: 两种经典 heap invariant

- 方案 A: 全部进大顶堆, 弹出 k-1 次, top 即第 k 大。
- 方案 B: 维护大小最多为 k 的小顶堆, 堆里始终保存“当前最大的 k 个”, top 是其中最小值, 也就是当前第 k 大。

```
std::priority_queue<int, std::vector<int>, std::greater<>> heap;
for (int x : nums) {
    heap.push(x);
    if (heap.size() > static_cast<size_t>(k)) heap.pop();
}
return heap.top();
```

★★ 自定义 priority_queue comparator 的方向

- priority_queue 的 comparator 表示“a 的优先级是否低于 b”。
- 因此最希望先出来的元素, 在 comp 意义下应被视为“最大”。
- 写完 comparator 后最好用 2-3 个手工样例验证 top。

★★ 合并 K 个有序数组

- 小顶堆中保存 value / arrayIndex / elementIndex。
- 初始把每个非空数组的第 0 个元素入堆。
- 每次弹出最小值, 再把该数组的下一个元素入堆。

9. Iterator、边界与失效规则

★ 首尾元素访问

- vector/list/deque 有 front() / back(); set 可 *begin() / *rbegin()。
- 任何解引用、front/back 前都先确认非空。
- end() 是尾后 iterator, 不能解引用。

★★ end() - 1 与 std::prev(end())

- end() - 1 只适用于 random access iterator (如 vector/deque/string)。
- std::prev(end()) 需要可向后移动的 bidirectional iterator; 不适用于 forward_list。
- forward_list 若要最后一个元素, 只能遍历或额外维护。

★★ 不同容器 iterator 永远不能混用

- 即使两个容器类型相同, iterator 也只能用于它所属的那个具体容器。
- 典型错误: a.erase(std::unique(b.begin(), b.end()), a.end())。

★★★ 常见 iterator 失效速查

- vector push_back: 若扩容则全部 iterator/reference/pointer 失效; 不扩容时 end() 失效。
- vector erase: 被删位置及其后的 iterator/reference 失效。
- list erase: 只有被删除元素的 iterator/reference 失效。
- map/set insert: 通常不会使已有 iterator/reference 失效。
- unordered_map rehash: iterator 失效; reference/pointer 通常仍指向元素。

关键习惯: 任何 insert / erase / push_back / rehash 之后, 先问“我手上的 iterator 还能不能用?”

★★ 修改容器前先保存 index，而不是事后用旧 iterator

- vector insert 可能扩容，使 pos 失效。
- 如果需要返回插入位置，先计算 distance/index，再 insert。

```
auto pos = std::lower_bound(v.begin(), v.end(), x);
auto index = std::distance(v.begin(), pos);
v.insert(pos, x);
return index;
```

10. C++20 / ranges 常用增强

★ std::erase / std::erase_if

- 把 erase-remove 模板收敛成一行，减少 iterator 错误。

★★ contains

- set / map / unordered_* 可用 contains(key)，比 find(key) != end() 更直接。

★★ ranges + projection

- 用 projection 让算法只看结构体某个字段，减少自定义 comparator 样板。

```
auto it = std::ranges::lower_bound(users, targetId,
    std::ranges::less{}, &User::id);
```

★★★ <=> 与 ordering 类型

- int 等全序类型通常得到 strong_ordering。
- double 因 NaN 只能形成 partial_ordering。
- 排序容器和 std::sort 仍要求可形成 strict weak ordering；NaN 数据需业务层处理。

11. 33 道维度 1 练习题索引

建议按“基础 -> 面试常用 -> 综合”三轮完成。

题号	难度	主题	训练重点
1	*	find_if	第一个满足复合条件的元素 + 下标
2	**	reverse_iterator	最后一个满足条件 + base() 转换
3	*	all/any/none_of	空范围语义
4	**	search	连续子序列 + 重叠匹配 + 循环进展
5	*	remove/erase	erase-remove 与 std::erase
6	*	remove_if	条件删除与保持顺序
7	**	unique	相邻去重 vs 全局去重
8	**	map::erase	安全遍历删除
9	**	sort comparator	多字段混合升降序
10	**	strict weak ordering	识别 <= comparator 错误
11	**	stable_sort	稳定性的真正语义
12	**	partial_sort/nth_element	Top-K 与第 k 个边界
13	*	lower/upper/equal_range	二分基础

题号	难度	主题	训练重点
14	**	insert position	lower/upper_bound + iterator 失效
15	**	descending lower_bound	降序 comparator 一致性
16	**	heterogeneous lookup	结构体按 key 二分 / projection
17	**	set::lower_bound	成员算法与通用算法复杂度
18	*	partition	奇偶分区与方向
19	**	two partitions	三段划分
20	**	nth_element	Quickselect 思维
21	**	set_intersection	排序前置条件与重复语义
22	*	unordered_set	保序去重
23	**	frequency sort	hash + vector + sort
24	**	interval comparator	start 升序、end 降序
25	**	Kth largest	大顶堆/小顶堆/nth_element
26	**	custom heap	priority_queue comparator
27	**	K-way merge	小顶堆 + 多数组索引
28	**	container ends	front/back/rbegin/prev/end-1
29	***	iterator invalidation	容器修改失效表
30	**	erase_if	四类容器安全删除
31	***	Order 清洗排序	过滤 + 分组 + 排序 + 二分
32	***	Market Top Levels	价格聚合 + Bid/Ask 排序 + Top5
33	***	代码审查	综合识别 comparator/iterator/前置条件问题

11.1 推荐练习顺序

第一轮：1、5、7、9、13、15、17、18、25、30。

第二轮：2、8、11、12、16、19、21、26、28、29。

第三轮：31、32、33。

12. 本轮练习暴露出的高频问题

这些问题比“会不会某个 STL API”更值得优先修正。

序号	问题类型	典型表现	优先级
1	Iterator 生命周期	vector insert 后继续用旧 pos; 未初始化 iterator; 不同容器 iterator 混用。	最高优先级
2	k 与下标边界	第 k 个是 begin+(k-1)，前 k 个区间终点是 begin+k。	最高优先级

序号	问题类型	典型表现	优先级
3	方向一致性	partition 奇偶方向、partial_sort 最大/最小、nth_element comparator。	高
4	二分前置条件	排序规则与 lower_bound comparator 必须一致。	高
5	循环进展	search 找不到时 next 不变会死循环。	高
6	stable_sort 语义	只比较 priority，不要把原顺序字段再次加入 key。	中高
7	取负降序	-INT_MIN 溢出；浮点特殊值使语义更复杂。	中
8	浮点校验	price <= 0 无法过滤 NaN；可用 !(price > 0) 或 isfinite。	中
9	空容器	front/back/*begin/*rbegin/prev(end) 前检查 empty。	高
10	unique 语义	unique 只处理相邻重复。	中
11	Heap invariant	先定义堆中到底维护“全部元素”还是“最大的 k 个”。	高
12	业务条件完整性	题目说 quantity<=0 无效时，不能只检查 price。	高

结论：当前知识覆盖已经够用。下一阶段收益最高的是：少扩 API，多做边界 / 前置条件 / iterator / 方向的刻意训练。

12.1 一眼识别的错误模式

错误模式	为什么错	修复习惯
修改后再算位置	insert/erase 之后继续使用旧 iterator	先计算 index 或使用返回的新 iterator
死循环	while 某个失败分支不改变 next/left/right	每条路径必须推进或退出
二分失配	数据降序但 lower_bound 默认 less	排序 comparator 与查找 comparator 保持一致
Top-K off-by-one	nth_element(begin+k)	第 k 个 1-based -> begin+(k-1)
稳定排序误用	stable_sort comparator 中加入 arrivalSequence	只比较真正 key
全局去重误用	直接 unique 未排序	先明确是否只去相邻、是否需保序

13. Tuple 多字段排序与方向包装

核心思想：把每个字段转换成一个“统一可按 < 比较的 key”，然后让 tuple 做字典序比较。

★ 全部升序：std::tie / std::tuple

- 字段全部按升序时最简单。

```
return std::tie(a.score, a.age, a.name)
    < std::tie(b.score, b.age, b.name);
```

★★ 简单混合方向：数值取负

- 例如 score 降序、age 升序、name 升序，可以把 score 取负。

- 但这只是简写，必须确认不会出现 INT_MIN 等溢出边界。

```
return std::tuple{-a.score, a.age, a.name}
    < std::tuple{-b.score, b.age, b.name};
```

风险：-INT_MIN 不可表示；对 double 还要考虑 NaN。生产代码或通用库不建议把“取负”当成通用降序方案。

*** Desc 包装器：把某字段的 < 方向反过来

- 通过包装类型让 tuple 仍统一使用 <，但某个字段内部按反方向比较。
- 避免整数取负溢出，也更清楚地表达“这一列降序”。

```
template <typename T>
struct Desc {
    const T& value;

    friend bool operator<(const Desc& lhs, const Desc& rhs) {
        return rhs.value < lhs.value;
    }
};

return std::tuple{Desc<int>{a.score}, a.age, a.name}
    < std::tuple{Desc<int>{b.score}, b.age, b.name};
```

*** C++20 <=> 版本

- 包装器可定义 operator<=>，把 rhs.value <=> value 作为降序比较。
- 对 double 会得到 partial_ordering，因此仍需定义 NaN 策略。

```
template <typename T>
struct Desc {
    const T& value;

    auto operator<=>(const Desc& rhs) const {
        return rhs.value <=> value;
    }
    bool operator==(const Desc& rhs) const {
        return value == rhs.value;
    }
};
```

** Living coding 推荐：显式字段比较通常最稳

- 优点：方向一眼可见、无取负溢出、容易加业务特殊规则、调试简单。

```
if (a.priority != b.priority) return a.priority > b.priority;
if (a.timestamp != b.timestamp) return a.timestamp < b.timestamp;
return a.id < b.id;
```

选择建议：简单竞赛题且范围明确：tuple + -x 可以很快；面试 / 生产代码：优先显式 comparator；通用封装：可用 Desc。

14. 综合题与面试 30 秒自检

14.1 综合题 A：订单清洗与排序

- 过滤 quantity <= 0。
- 过滤非正价格；若 price 为 double，明确 NaN / infinity 策略。
- 按 symbol 分组。
- 组内 price 降序，timestamp 升序。
- 如果随后用二分查找，查找 comparator 必须与组内顺序一致。

浮点校验模板：若目标是“只允许有限正数”，建议 !std::isfinite(price) || price <= 0.0。若题目只要求正数，也可用 !(price > 0.0) 顺便排除 NaN。

14.2 综合题 B: Market Data Top Levels

6. 删除无效价格和 quantity <= 0 的记录。
7. 相同 scaledPrice 聚合 quantity。
8. Bid 用 greater 价格降序；Ask 用 less 价格升序。
9. 只保留前 5 档。
10. 明确 scaledPrice 的单位和溢出边界。

14.3 面试 30 秒自检清单

- ☐ 01. 输入可能为空吗？front/back/解引用是否安全？
- ☐ 02. k 合法吗？k 是 1-based 还是 0-based？
- ☐ 03. 第 k 个位置和前 k 个区间是否区分清楚？
- ☐ 04. 默认算法方向是升序/最小优先，还是题目要求降序/最大优先？
- ☐ 05. 修改容器后，手上的 iterator/reference/pointer 是否失效？
- ☐ 06. while/for 的每条路径是否推进或退出？
- ☐ 07. lower_bound / upper_bound 的范围是否按同一 comparator 排好？
- ☐ 08. comparator 是否严格？cmp(x,x) 是否为 false？
- ☐ 09. 是否用了 -x，可能遇到 INT_MIN / 浮点特殊值？
- ☐ 10. unique 前是否真的只需要相邻去重？
- ☐ 11. heap 中维护的 invariant 能否用一句话说清楚？
- ☐ 12. 是否漏掉题目中的过滤条件、字段或边界？

14.4 最终复习优先级

优先级	目标	内容
P0	必须形成条件反射	iterator 失效、空容器、k 下标、二分 comparator 一致性
P1	面试高频	sort comparator、stable_sort、heap invariant、unique/erase-remove
P2	提升代码表达	ranges projection、erase_if、tuple 多字段比较
P3	进阶严谨性	NaN、三路比较、generic Desc wrapper、iterator category 复杂度

一句话总结

维度 1 的目标不是“记住更多 STL”，而是把每一个 STL 调用的前置条件、边界、方向和 iterator 语义写对。